Earned Autonomy Wallet + Financial Advisor

> আপনি বিল আপলোড করবেন → AI যাচাই করে payment প্রস্তুত করবে → আপনার নির্দিষ্ট লেনদেনে passkey/fingerprint অনুমোদন → bank/PSP-এর অনুমোদিত সংযোগ থাকলে অ্যাপ নিজে payment request পাঠাবে। অন্য ধাপগুলো অ্যাপ সামলাবে।

AI কী করে

বিল পড়ে, খরচের তালিকা আপডেট করে, মাসিক রিপোর্ট দেয়; বিশ্বাস অর্জনের পর bill payment প্রস্তুত করে। অনুমোদনের পর সংযুক্ত provider-এ request পাঠায়।

AI কী পারে না

অনুমতি ছাড়া ব্যাংকে request পাঠায় না। সন্দেহজনক লেনদেন সতর্ক করে; বাহিরের card payment থামাতে issuer-এর অনুমোদিত control লাগে।

এই সংশোধনের মূল নিয়ম

বিশ্বাস অর্জন = নিয়মিত বিল **নিজে প্রস্তুত, সময়মতো উপস্থাপন ও অনুমোদনের পরে নিজে জমা** দেওয়ার অধিকার। প্রতিটি পেমেন্টের চূড়ান্ত user verification বাধ্যতামূলক।

> **ডেমো সীমা:** এখনকার prototype-এ demo bank/PSP ও card feed থাকবে; আসল টাকা যাবে না। বাস্তব সংস্করণে লিখিত bank/PSP/issuer অনুমতি, API, চুক্তি ও compliance লাগবে।

মূল ১০-পৃষ্ঠার গাইডের Advisor, বিল-ব্যবস্থাপনা, Wilson lower bound, Guardian, audit, শুরু থেকে বানানোর ধাপ এবং ন্যূনতম ২০টি পরীক্ষা বজায় রেখে এই সংস্করণ তৈরি। ১৮ অক্টোবর ২০২৬-এ যাচাই করা তথ্যের ভিত্তিতে প্রতিযোগিতার অবস্থান লেখা হয়েছে।

১ · বিষয় নির্বাচন ও সত্য দাবি

বিষয়	এই প্রকল্পের অবস্থান
Topic F Agent Wallet / Autonomous On-chain Wallet	প্রকাশিত বিবরণে এটি স্পষ্টত **on-chain** বিষয় [১]। এই নকশায় কোনো blockchain বা real on-chain transaction hash নেই। তাই মূল গাইডে উল্লিখিত on-chain hash শর্ত পূরণ হয়েছে বলে দাবি করা যাবে না। বিস্তারিত নিয়মে hash-এর ভাষা আলাদাভাবে যাচাই করতে হবে।
Topic C Cross-border & Student Finance Assistant	শুধু বাংলাদেশি বিল ও ওয়ালেট হলে উপযুক্ত নয়। প্রকৃত আন্তর্জাতিক শিক্ষার্থী/সীমান্তপার অর্থব্যবস্থার সমস্যা, ব্যবহারকারী গবেষণা ও বৈধ সমাধান থাকলে বিবেচ্য; নইলে আয়োজকের লিখিত ব্যাখ্যা চাইতে হবে [১]।
Topic A Personal Finance Assistant	খরচ বিশ্লেষণ, বিল ও সাশ্রয়ের পরামর্শের জন্য প্রকাশিত বর্ণনার সঙ্গে সবচেয়ে সরাসরি মেলে। তবে জমার ফর্মে কোন বিষয় গ্রহণযোগ্য তা আয়োজককে নিশ্চিত করা ভালো [১]।

জমার সময় ও যোগ্যতা

আন্তর্জাতিক ট্র্যাকের প্রকাশিত সময়সূচি অনুযায়ী **২০ অক্টোবর ২০২৬** নিবন্ধন ও জমার শেষ তারিখ, সময় অঞ্চল UTC+8; নির্দিষ্ট ঘড়ির সময় প্রকাশিত সারাংশে নেই। আগেই জমা দিন। ২–৫ সদস্য; সবাই চীনের বাইরে বিশ্ববিদ্যালয়ের পূর্ণকালীন শিক্ষার্থী; অন্তত ৬০% অ-চীনা নাগরিক [১]।

> **ভাষা:** আন্তর্জাতিক ট্র্যাকের দাপ্তরিক ভাষা ইংরেজি [১]। এই বাংলা গাইড দিয়ে কাজ তৈরি করুন; চূড়ান্ত ডেক/বর্ণনা/ডেমো ইংরেজিতে প্রস্তুত করুন।

[১] IEEE CIS Singapore, “2026 FinTechathon”, ieee-cis-singapore.org/2026-fintechathon (১৮ অক্টোবর ২০২৬ দেখা)।

২ · ওয়ালেট, অনুমোদন ও ভূমিকা

ওয়ালেট বলতে এখানে কী বোঝায়?

প্রথম সংস্করণে wallet হলো demo হিসাবখাতা। বাস্তব সংস্করণে টাকা আপনার ব্যাংকেই থাকবে; অ্যাপ অনুমোদিত payment-initiation connector হবে, নিজে bank balance-এর মালিক নয়। App receipt, bank submission ID এবং bank-confirmed settlement আলাদা status। SIM receipt কোনো bank receipt বা on-chain hash নয়।

অংশ	কাজ	পেমেন্ট সম্পাদনের ক্ষমতা
Advisor / AI Agent	CSV ও বিল থেকে তথ্য, পরামর্শ, খসড়া	নেই
Scheduler	বিলের তারিখ দেখে কাজ প্রস্তুত ও সতর্কতা	নেই
Guardian	স্থির নিয়ম, সীমা, বিশ্বাস, ঝুঁকি যাচাই	শুধু অনুমোদনযোগ্য কি না বলে
WebAuthn verifier	এই নির্দিষ্ট পেমেন্টে নতুন user verification যাচাই	নিজে লেজার বদলায় না
Executor / bank adapter	ডেমোতে ledger; live-এ অনুমোদিত provider request	শুধু যাচাইকৃত সার্ভার

পেমেন্টের যাত্রা

বিল আপলোড → AI extraction → Guardian → approval notification → বিস্তারিত + passkey → provider request → provider confirmation → খরচের তালিকা + audit

সততা: "ফিঙ্গারপ্রিন্ট" UI-তে লক্ষ্য হতে পারে, কিন্তু সাধারণ WebAuthn সার্ভার জানে *user verification* হয়েছে; সেটি আঙুল, ডিভাইস PIN, বা অন্য স্থানীয় পদ্ধতি ছিল কি না সাধারণত জানে না [২]। এই গাইডে ডিভাইস PIN fallback গ্রহণযোগ্য। তাই "প্রতি পেমেন্টে verified passkey approval" দাবি করুন, "ফিঙ্গারপ্রিন্ট ছাড়া কখনো সম্ভব নয়" দাবি নয়।

৩ · সম্পূর্ণ আর্কিটেকচার

React / Vite ওয়েব UI
Dashboard · Advisor · Bills · Trust · Approvals · History

↓ HTTPS + session + CSRF protection

FastAPI সার্ভার
Bill parser · Agent · Scheduler · Trust · Guardian · Risk engine · WebAuthn · Provider adapter

↓ শুধু যাচাইকৃত সার্ভার কোড

SQLite ডেমো ডেটাবেস
transactions · bills · proposals · trust_events · approvals · provider_events · card_events · audit

দুটি স্পষ্ট সীমানা

1. **AI → Guardian:** AI-এর লেখা শুধু অবিশ্বস্ত প্রস্তাব। ব্যাংক অ্যাকাউন্ট, প্রাপক, অঙ্ক ও তারিখ সার্ভার যাচাই করে; AI সরাসরি executor ডাকতে পারে না।
2. **Guardian → Executor:** Guardian "অনুমোদনযোগ্য" বললেও লেনদেন *pending* থাকে। ব্যবহারকারীর ওই লেনদেনের জন্য নতুন WebAuthn assertion না এলে executor চলবে না।

অবস্থার ধাপ

DRAFT → CHECKED → PENDING_USER → AUTHORIZED → SUBMITTED → CONFIRMED
অথবা HOLD / DECLINED / UNKNOWN / EXPIRED; demo-তে SIMULATED_PAID

Bank connector শুধু অনুমোদিত provider-এর documented API ব্যবহার করবে। Card feed ও দ্বিতীয় ডিভাইস alert আলাদা service; কোনো login password, CVV বা raw fingerprint অ্যাপে নয়। সার্ভারের state machine ছাড়া UI বোতাম দিয়ে অবস্থা পাল্টানো যাবে না। প্রতিটি transition-এর সময়, actor, কারণ ও পুরোনো/নতুন অবস্থা audit-এ লিখুন।

৪ · খরচ বুঝে পরামর্শ

ইনপুট

প্রথমে ব্যবহারকারী নিজের bank/MFS statement-এর CSV বা হাতে খরচ লিখবে। অনুমোদিত bank/issuer feed থাকলে debit/credit card purchase event নিজে আসবে এবং খরচের তালিকায় যোগ হবে। CSV-তে তারিখ, বিবরণ, অঙ্ক, debit/credit-এর কলামভেদ থাকতে পারে। আগে preview দেখিয়ে ব্যবহারকারীকে কলাম মেলাতে দিন; নাম/ফোন/অ্যাকাউন্ট নম্বর mask করুন। ব্যক্তিগত statement AI provider-এ পাঠানোর আগে স্পষ্ট সম্মতি নিন; সম্ভব হলে সারাংশই পাঠান।

হিসাবের ধাপ

1. CSV-এর encoding ও date format যাচাই; ভুল সারি আলাদা error report।
2. মুদ্রা **Decimal** বা integer poisha-তে রূপান্তর। float-এ টাকার হিসাব নয়।
3. কীওয়ার্ড নিয়মে খাবার, বিল, ভাড়া, যাতায়াত, subscription ইত্যাদি শ্রেণি; অনিশ্চিত হলে "review"। AI শ্রেণি প্রস্তাব দিতে পারে, চূড়ান্ত সত্য নয়।
4. একই প্রাপক + কাছাকাছি অঙ্ক + প্রায় মাসিক ব্যবধান দিয়ে পুনরাবৃত্ত খরচ শনাক্ত। ভুল subscription ব্যবহারকারী ঠিক করতে পারবে।
5. মাস শেষে ব্যাংক, debit/credit card ও manual খরচ একত্রে reconciliation করে মাসিক মোট, category, budget variance, refund ও credit-card repayment-এর double count বাদ দিয়ে হিসাব করুন। AI ফল ব্যাখ্যা করবে; খরচ কমানো/বাড়ানোর বিকল্প ও দক্ষতা/অতিরিক্ত কাজের বাস্তবসম্মত আয়ের ধারণা দেবে, নিশ্চয়তা নয়।

> **উদাহরণ:** "তিনটি streaming subscription-এ মাসে ৳৮০০, বছরে ৳৯,৬০০। কোনটি প্রয়োজন তা দেখুন।" সংখ্যা হিসাব থেকে এসেছে; AI বানায়নি।

> **মাসশেষের নমুনা:** "এই মাসে খাবার ৳১,৫০০ বেড়েছে; delivery কমালে আনুমানিক ৳৬০০ সাশ্রয় হতে পারে। দক্ষতা বাড়াতে কোর্সে খরচ বাড়ানো যুক্তিযুক্ত কি না দেখুন। আপনার সময়/দক্ষতার সঙ্গে মেলে এমন tutoring বা freelance কাজ পরীক্ষা করতে পারেন।" সব অঙ্ক verified data থেকে; আয়ের ধারণা নিশ্চয়তা নয়।

এটি ব্যয় বোঝার সহায়তা; আয় বৃদ্ধির ধারণা সম্ভাবনা হিসেবে দিন, আয় বা বিনিয়োগের নিশ্চয়তা নয়। ব্যক্তিগত বিনিয়োগ বা লাইসেন্সপ্রাপ্ত আর্থিক পরামর্শ নয়। ডেমো ডেটা দিয়ে শুরু করুন, পরে সম্মতি নেওয়া বাস্তব CSV যোগ করুন।

৪.১ · Card shopping থেকে auto expense

আপনার card দিয়ে দোকানে payment হলে

ব্যাংক/issuer-এর সম্মতিপ্রাপ্ত transaction feed বা webhook থাকলে purchase event আসবে। অ্যাপ merchant, সময়, মুদ্রা, অঙ্ক ও card-এর masked token দেখিয়ে expense list-এ যোগ করবে; user চাইলে category ঠিক করতে পারবে। একটি card নম্বর "connect" লিখলেই সব ব্যাংকের live feed পাওয়া যায় না। Direct feed না থাকলে statement CSV/অনুমোদিত export দিয়ে শুরু করুন। Transaction feed পাওয়া মানেই দোকানের payment approval নিয়ন্ত্রণ নয়।

Card authorization (pending) → issuer feed → expense preview
Posted charge / refund → reconciliation → final monthly totals

দুইবার গণনা ঠেকান

- Provider event ID + account token দিয়ে dedupe; pending ও posted একই purchase-এর দুই অবস্থা।
- Refund/chargeback মূল খরচের সঙ্গে link; credit-card bill পরিশোধ **transfer**, দ্বিতীয় expense নয়।
- বিদেশি মুদ্রায় মূল currency ও bank-converted BDT আলাদা রাখুন; অনুমানকৃত exchange rate দিয়ে final total নয়।
- Bank data না এলে "sync delayed" দেখান; পূর্ণ মাসিক হিসাব দাবি নয়।

> **গোপনীয়তা:** raw card number, CVV, OTP, bank password অ্যাপে রাখবেন না। Partner-issued token, প্রয়োজনীয় transaction fields, user consent ও revocation রাখুন। Visa-র issuer card-control পণ্যেও alert/block feature issuer-এর সিদ্ধান্ত ও enrollment-এর ওপর নির্ভর করে [৪]।

৪.২ · সব card payment-এ fingerprint?

লেনদেন	আপনার অ্যাপে fingerprint gate সম্ভব?
বাসায় online Visa checkout	Issuer/3-D Secure/Click to Pay যদি আপনার app-কে অনুমোদিত authentication channel হিসেবে নেয়, issuer-নির্বাচিত challenge-এ fingerprint/PIN prompt সম্ভব। প্রতিটি checkout-এ challenge হবে, এমন নিশ্চয়তা নেই; সিদ্ধান্ত issuer-এর।
দোকানের POS/contactless Visa	সাধারণ তৃতীয় পক্ষের অ্যাপ দিয়ে প্রতিটি authorization থামিয়ে নিজস্ব fingerprint-এর অপেক্ষা করানো যায় না। Issuer/processor-এর real-time authorization architecture, scheme rules ও সময়সীমার মধ্যে অনুমোদিত integration থাকলে নির্দিষ্ট card program-এ নকশা করা যেতে পারে।
ATM withdrawal	ATM/issuer-এর PIN ও নিজস্ব নিরাপত্তা flow চলে; এই অ্যাপের fingerprint দিয়ে universal replacement নয়।

যা এখনই বাস্তবসম্মত

1. Prototype-এ fake Visa authorization request → app notification → fingerprint/PIN → issuer mock approve/decline। এটি **simulation** লিখুন।
2. বাস্তব bank/issuer feed থাকলে successful card payment খরচের তালিকায় দ্রুত যোগ ও unknown merchant/amount alert। Notification payment-এর *পরে* এলেও তা fraud response-এ কাজে লাগে।
3. যদি issuer শুধু card on/off বা category block দেয়, আগে থেকে rule সেট করা যায়; প্রতিটি POS transaction-এ ad hoc fingerprint prompt দাবি করা যাবে না।

> Card payment-এর শেষ অনুমোদন issuer দেয়। Visa-র [authorization flow] তা দেখায়; [3DS issuer-app challenge] online shopping-এর উদাহরণ; [Visa Transaction Controls] issuer-নির্বাচিত alert/block।

৫ · বিল ও সময়সূচি

বিলের তথ্য: bill ID, অনুমোদিত প্রাপক ID, অঙ্ক, currency=BDT, শেষ তারিখ, invoice/reference, source, status। একই প্রাপক+reference+period আবার এলে duplicate হিসেবে থামান। ছবির OCR/AI তথ্য বের করলেও ব্যবহারকারী/নিয়ম দিয়ে যাচাই হবে।

সময় (উদাহরণ: ৭ অক্টোবর শেষ তারিখ)	কাজ
১০ দিন আগে	বিল শনাক্ত; প্রাপক, অঙ্ক, duplicate ও reserve যাচাই শুরু।
৭ দিন আগে	Guardian review; trust tier অনুসারে AI খসড়া নিজে প্রস্তুত করতে পারে।
৩ দিন আগে	অনুমোদন না থাকলে ব্যবহারকারীকে অনুরোধ ও reminder।
৫ অক্টোবর: safe date	অনুমোদন থাকলে demo execution অথবা অনুমোদিত provider request; bank confirmation না এলে "paid" নয়। না থাকলে "approval needed" সতর্কতা।
৭ অক্টোবর	বাকি থাকলে জরুরি সতর্কতা; কোনো নীরব auto-pay নয়।

সূত্র: safe_date = due_date − buffer_days; buffer ব্যবহারকারী ঠিক করবেন। Scheduler অ্যাপ বন্ধ থাকলেও চলার জন্য আলাদা scheduled worker/cron-এ রাখুন। একই bill বারবার চালালেও একটিই proposal/receipt হবে।

> **গুরুত্বপূর্ণ:** পূর্বে biometrics দিয়ে "সব ভবিষ্যৎ বিল" অনুমোদন নয়। প্রতিটি বিল/লেনদেনের নির্দিষ্ট প্রাপক, অঙ্ক ও reference দেখিয়ে নতুন WebAuthn অনুমোদন লাগবে। ব্যবহারকারী সাড়া না দিলে pending থাকবে, টাকা যাবে না। Fingerprint দেওয়ার পর অ্যাপ নিজে provider request পাঠাবে; provider-এর নিজস্ব OTP/redirect থাকলে তা এড়ানো যাবে না।

৬ · Earned Autonomy ও Wilson lower bound

প্রতিটি কাজের ধরনের আলাদা বিশ্বাস রাখুন: নিয়মিত বিদ্যুৎ বিল, subscription, নতুন প্রাপক, বড় অঙ্ক। Shadow mode-এ AI "আমি এই খসড়া করতাম" বলে; ব্যবহারকারীর সিদ্ধান্তের সঙ্গে ফল মিলিয়ে গণনা হয়। প্রস্তাবের পর পাওয়া সত্যিকারের সিদ্ধান্তই live evidence।

$$\hat{p} = s/n;\ L = [\hat{p} + z^2/(2n) - z\sqrt{(\hat{p}(1-\hat{p})/n + z^2/(4n^2))}] / [1 + z^2/n],\ z = 1.96$$

এখানে n = যাচাইকৃত সিদ্ধান্তের সংখ্যা, s = নিরাপদ/সঠিক সিদ্ধান্তের সংখ্যা। 95% Wilson lower bound **L** ছোট নমুনায় অতিরিক্ত আত্মবিশ্বাস কমায়। n=0 হলে L=0।

Tier	উদাহরণ নীতি	কী স্বাধীনতা
0	শুরু / unsafe error	শুধু পরামর্শ; ব্যবহারকারী হাতে খসড়া নিশ্চিত।
1	n≥20, L≥0.80	চেনা বিলের খসড়া ও reminder নিজে বানায়।
2	n≥40, L≥0.90	চেনা বিলের safe date-এ pending approval queue বানায়।
3	n≥80, L≥0.95	চেনা প্রাপকের সীমার মধ্যে বিল নিজে প্রস্তুত করে; fingerprint approval-এর পরে bank request নিজে পাঠায়। নতুন প্রাপক/অঙ্ক/সন্দেহে অতিরিক্ত review।

২০/২০ হলে L≈0.839, ৪০/৪০ হলে ≈0.912, ৮০/৮০ হলে ≈0.954; তাই মূল গাইডের milestone বজায় থাকে। Threshold প্রকল্পের নীতি, প্রতিযোগিতার শর্ত নয়। Tier কেবল AI-কে **request প্রস্তুত/পাঠানোর workflow** দেয়; অনুমোদন ছাড়া provider call নয়।

> **Unsafe ভুল:** AI যে লেনদেন করা উচিত ছিল না সেটি করার প্রস্তাব দিলে ধরনটির tier 0, pending proposal বাতিল, audit ও review। পুরোনো ৬–১২ মাসের replay ডেমোর evidence হতে পারে; outcome যাচাই, data leakage এড়ানো ও live evidence আলাদা দেখান। Replay একা production trust প্রমাণ করে না।

৭ · নিয়ম ও অনুমতি

Guardian AI নয়: পরীক্ষা-যোগ্য সাধারণ সার্ভার কোড। Bank/issuer data থেকে পাওয়া ঝুঁকি সংকেতও নেবে। প্রত্যেক check `(pass, reason)` ফেরত দেয়; যে কোনো ব্যর্থতায় proposal থামে। "Illegal" বলে আইনগত রায় নয়; **সন্দেহজনক/সম্ভাব্য অননুমোদিত** বলে সতর্ক করবে। AI লেখা পরিবর্তন করে নিয়ম শিথিল করতে পারবে না।

Check	ন্যূনতম আচরণ
প্রাপক	allowlist ID ও প্রদর্শিত নাম মেলে; নতুন/বদলানো প্রাপক user review।
অঙ্ক	positive integer poisha; ঐ প্রাপকের ছয় মাসের baseline-এর 2× ছাড়ালে hold; global hard cap।
বিল	reference+period duplicate নয়; due date যুক্তিযুক্ত; উৎস নথিভুক্ত।
তহবিল	demo balance ≥ amount এবং জরুরি reserve অবশিষ্ট।
বিশ্বাস	ধরন ও tier কেবল প্রস্তুতির অধিকার দেয়; final approval সব tier-এ দরকার।
সময়	expired/overdue হলে auto-preparation থামে; ব্যবহারকারীকে জানায়।
Injection	বিলের "সব নিয়ম উপেক্ষা কর" লেখা data হিসেবে থাকে; policy নয়। সন্দেহে hold + audit।

> **ভালো নকশা:** Guardian শুধু structured, server-validated bill record পড়ে। AI response বা OCR-এর raw instruction executor-এর কাছে যায় না। একই check approval-এর আগে এবং provider submission-এর ঠিক আগে আবার চালান। সন্দেহজনক হলে অন্য নিবন্ধিত ডিভাইসে challenge; সেখান থেকে "এটা আমার নয়" এলে app-initiated payment block। বাইরের card payment থামাতে issuer control integration অপরিহার্য।

৭.১ · Alert, অন্য ডিভাইস ও block

"Illegal transaction" শব্দটি আইনি সিদ্ধান্ত বোঝায়। অ্যাপ **সন্দেহজনক বা সম্ভাব্য অননুমোদিত** লেনদেন শনাক্ত ও ব্যাখ্যা করবে; অপরাধ নির্ধারণ করবে না। Risk score-এর কারণ দেখাবে: নতুন প্রাপক, অস্বাভাবিক অঙ্ক, নতুন device, আগের আচরণ থেকে বিচ্যুতি, duplicate বা নিষিদ্ধ merchant category।

কোথায় লেনদেন শুরু	আপনি "আমার নয়" বললে
এই অ্যাপ থেকে bill payment	Guardian HOLD → অন্য নিবন্ধিত ডিভাইসে exact transaction challenge → deny-তে pending request বাতিল; provider-এ এখনও পাঠানো হয়নি।
বাইরের দোকানে debit/credit card	Bank/issuer event পেলে alert + report/freeze request। Card authorization আগে থেকেই আটকাতে issuer-এর real-time authorization flow/চুক্তি লাগবে; সাধারণ alert API যথেষ্ট নয়; না থাকলে কেবল পরের সতর্কতা।

Gmail-ধাঁচের যাচাই: আগে enrol করা দ্বিতীয় ফোনে push খুললে merchant/payee, অঙ্ক ও সময় দেখাবে। সেখানে passkey দিয়ে "আমার" বা "আমার নয়"। Push-এর token, device ownership, expiry ও single-use challenge যাচাই। সন্দেহ বেশি হলে দ্বিতীয় ডিভাইসের সাড়া না পাওয়া পর্যন্ত payment fail-closed।

> "Block" ফলের পাশে উৎস লিখুন: **App blocked**, **Bank/issuer block requested**, অথবা **Issuer confirmed blocked**। প্রথমটিকে তৃতীয়টি বলে দেখাবেন না।

৮ · প্রতিটি পেমেন্টে WebAuthn

1. ব্যবহারকারীকে **প্রাপক, ৳ অঙ্ক, বিল reference, fee=৳০ (ডেমো), due date** দেখান। "Approve this simulated payment" বোতাম চাপার পরেই WebAuthn শুরু।
2. সার্ভার cryptographically random, একবার ব্যবহারযোগ্য challenge দেয়। সার্ভারে তার সঙ্গে user ID, proposal ID, **canonical transaction hash**, expiry এবং status সংরক্ষণ করে।
3. ব্রাউজার `navigator.credentials.get()` কল করে `userVerification: "required"` সহ platform passkey ব্যবহার করে। ফোন/ল্যাপটপ স্থানীয়ভাবে fingerprint বা device PIN চাইতে পারে। অনুমোদিত হলে আর আলাদা "Pay" বোতাম নেই; server নিজে provider submission queue-তে পাঠায়।
4. সার্ভার signature, challenge, origin, RP ID, credential ownership, UV flag, expiry ও replay যাচাই করে। এরপর DB transaction-এ proposal-এর বর্তমান তথ্যের hash আবার মেলায় এবং challenge consume করে।
5. Guardian পুনরায় পাস হলে demo ledger লেখে অথবা অনুমোদিত bank connector-এ একবারই request পাঠায়। "Submitted" ও "Bank confirmed" আলাদা রাখে। ব্যর্থ/বাতিল/timeout-এ কিছুই debit হয় না।

> **বায়োমেট্রিক তথ্য:** ওয়েব অ্যাপ fingerprint ছবি বা template নেয়/রাখে না; authenticator স্থানীয় verification করে, অ্যাপ public key ও signed assertion দেখে [২]। PIN fallback-কে একই user verification flow হিসেবে নথিভুক্ত করুন।

সাধারণ WebAuthn assertion নিজে payment details বিশ্বস্ত পর্দায় দেখায় না। অ্যাপ UI-তে বিস্তারিত দেখানো ও সার্ভারে challenge→transaction binding দরকার। বাস্তব bank/PSP-এর নিজস্ব OTP, 3-D Secure বা অন্য অনুমোদন নীতি থাকতে পারে; অ্যাপের passkey একাই bank-এর শর্ত প্রতিস্থাপন করবে ধরে নেওয়া যাবে না। HTTPS (localhost উন্নয়নে ব্যতিক্রম), স্থির RP ID ও origin কনফিগার করুন।

৯ · সিমুলেটেড লেজার ও audit

একবারই debit

সব টাকা integer poisha। প্রতিটি payment-এর একটি unique idempotency key = proposal ID। একই অনুরোধ দুবার এলে আগের receipt ফেরত দিন। DB transaction-এর ভেতর balance check, debit ও matching credit, state update, approval consume, audit write একসঙ্গে করুন; মাঝখানে failure হলে সব rollback।

Wallet liability −৳১,২০০ ↔ Demo merchant liability +৳১,২০০
receipt: SIM-2026-00042 · status: simulated_paid

এটি হিসাবের ডেমো, ব্যাংক settlement নয়। বাস্তব bank feed-এ bank transaction ID ও settlement status-ই সত্যের উৎস; app-এ শুধু mirror/reconciliation। আলাদা demo fund top-up actor ছাড়া balance বদলানো যাবে না। কোনো negative balance নয়। প্রতিদিন ledger sum=0 ও wallet balance reconciliation চালান।

Audit event-এ রাখুন

timestamp (UTC), actor, action, proposal ID, bill ID, version, policy result ও reasons, trust tier/L, challenge ID (secret নয়), credential ID-এর নিরাপদ reference, receipt ID, prev_hash, event_hash। Hash chain স্থানীয় পরিবর্তন ধরতে সহায়ক; **এটি blockchain বা অপরিবর্তনীয় প্রমাণ নয়**। ব্যাকআপ/অ্যাক্সেস নিয়ন্ত্রণ প্রয়োজন।

> **প্রতিযোগিতার evidence:** SIM receipt, mock provider response, Guardian log ও test run সিমুলেশনের প্রমাণ। এগুলোকে on-chain transaction hash বলে জমা দেবেন না।

৯.১ · আসল ব্যাংক payment-এর জীবনচক্র

আপনার চাওয়া “একবার fingerprint, তারপর নিজে pay”

বিশ্বাস-সীমা পাস করা পরিচিত বিলের জন্য AI অঙ্ক/প্রাপক/তারিখ সাজাবে; push খুলে আপনি বিস্তারিত দেখবেন ও passkey verification দেবেন। এরপর অ্যাপ আর কোনো নিজস্ব “Pay” চাপা ছাড়াই অনুমোদিত bank/PSP payment API-তে request পাঠাবে। Provider নিজস্ব OTP, redirect বা bank-app confirmation চাইলে সেটিও করতে হবে; অ্যাপ তা বাদ দিতে পারে না।

PREPARED → USER_VERIFIED → SUBMITTED → BANK_CONFIRMED → RECONCILED
অথবা HOLD / DECLINED / UNKNOWN / REVERSED

Bank connector contract: `submit(account_token, payee_id, amount_poisha, bill_ref, idempotency_key)`, `get_status(provider_ref)`, verified webhook। কেবল consented token; bank password/OTP/CVV সংরক্ষণ নয়। Provider reference ছাড়া app-generated receipt দিয়ে “paid” দেখানো নয়। Timeout হলে status query; অনিশ্চিত ফলের পরে নতুন key দিয়ে পুনরায় পাঠিয়ে double pay নয়।

পর্যায়	কী সম্ভব
Prototype	mock bank API + demo ledger: পুরো UX ও নিরাপত্তা দেখানো।
Partner sandbox	bank/PSP পরীক্ষার API থাকলে end-to-end request ও callback।
Live	চুক্তি, provider অনুমতি, bank authentication, নিরাপত্তা ও প্রযোজ্য লাইসেন্স/নিয়ম যাচাইয়ের পর।

বাংলাদেশ ব্যাংকের PSP/PSO অনুমোদন কাঠামো ও ২০২৪ সালের payment law-এর প্রেক্ষাপটে শুধু public API খুঁজে পাওয়া live payment চালুর অনুমতি নয় [৩]। নির্দিষ্ট ব্যাংকের contract ও regulatory counsel দিয়ে পথ যাচাই করুন।

৯.২ · Bank account/ATM validation কীভাবে

> **সবচেয়ে জরুরি উত্তর:** bank account number, branch বা card-এর শেষ চার সংখ্যা জানলেই কোনো অ্যাপ transaction দেখতে, টাকা কাটতে বা Visa authorization থামাতে পারে না। এগুলি শনাক্তকারী তথ্য, অনুমতির চাবি নয়। আপনার নিজের account হলেও ব্যাংকের অনুমোদিত সংযোগ ও পরিচয় যাচাই দরকার।

নিরাপদ account link-এর ধাপ

1. ব্যাংক/অনুমোদিত provider-এর সঙ্গে API ও permitted scope নির্ধারণ: transaction **read**, payment **initiate**, card alert/control - তিনটি আলাদা ক্ষমতা। সব bank-এ সব scope নাও থাকতে পারে।
2. ব্যবহারকারীকে bank-এর নিজস্ব app/website বা provider-approved flow-তে পাঠান। সেখানে bank OTP/2FA/অন্যান্য পদ্ধতিতে account ownership ও consent যাচাই করে; অ্যাপ bank login password নেয় না।
3. Bank app-কে সীমিত মেয়াদের token/account reference দেয়। কেবল consented scope ব্যবহার করুন; revoke, expiry, device change ও lost-phone recovery রাখুন। Provider signed event/webhook যাচাই করুন।
4. Card feed থাকলে purchase import; payment-initiation scope থাকলে ৳৫০০ transfer বা bill request; issuer authorization scope থাকলে শুধু চুক্তির অধীনে card control। একটি scope অন্যটির বদলি নয়।

“ATM validation” বলতে যদি ATM/debit card বোঝান

ATM PIN, CVV, full card number বা bank OTP আপনার অ্যাপে লিখবেন না। ATM cash withdrawal ও দোকানের POS card authorization ব্যাংক/issuer-এর card network flow-তে হয়; নিজের passkey দিয়ে ATM PIN-এর জায়গা নেওয়া যায় না। Bank partner যদি আলাদা cardholder verification/product দেয়, কেবল তার documented flow বাস্তবায়ন করুন।

Bangladesh Bank-এর PSP/PSO approval procedure ও Payment Systems Report ব্যাংকিং/পেমেন্ট সেবায় অনুমোদন কাঠামো দেখায়: BB approval procedure · BB 2025 report। নির্দিষ্ট ব্যাংক ও চুক্তির শর্ত আলাদা যাচাই করুন।

১০ · ফোল্ডার, সরঞ্জাম ও প্রথম সেটআপ

```
earned_autonomy/
  backend/
    main.py               # FastAPI routes
    models.py             # typed input/output
    ledger_db.py          # SQLite schema + atomic ledger
    advisor.py            # Decimal-based spending analysis
    bills.py              # due date + scheduler
    autonomy.py           # Wilson score + per-category tier
    checks.py             # deterministic Guardian rules
    agent.py              # optional AI extraction/explanation
    webauthn_service.py   # registration + transaction approval
    audit.py              # append-only audit events
    worker.py             # scheduled reminders / proposals
    bank_connector.py     # licensed partner adapter only
    card_feed.py          # consented issuer events
    risk.py               # suspicious activity rules
    device_push.py        # registered second device
    monthly_report.py     # reconciliation + advice
  frontend/               # React + Vite WebAuthn UI
  tests/                  # policy, flow, replay, ledger tests
  demo_data/              # synthetic CSV and bills
  .env.example  .gitignore  README.md
```

মূল গাইড থেকে রূপান্তর: `chain.py`, MetaMask, Sepolia, RPC, private key ও `web3.py` বাদ। `ledger_db.py` এখন payment simulation চালায়। আগের `autonomy.py`/`sim.py` থাকলে বাস্তবে খুলে পরীক্ষা করে পুনর্ব্যবহার করুন; ফাইল আছে ধরে নেবেন না।

1. Python 3.10+, Node.js, Git, VS Code; নতুন repo তৈরি।
2. Python virtual environment; FastAPI, Uvicorn, pytest, pandas, WebAuthn server library install করুন। লাইব্রেরির বর্তমান API অনুযায়ী উদাহরণ মিলিয়ে নিন।
3. Vite React app; browser WebAuthn API দিয়ে কাজ। `.env` Git-এ নয়; demo signing key নয়, শুধু optional AI API key।

১১ · ধাপে ধাপে: তথ্য থেকে খসড়া

ধাপ ১ · Database: transactions, bills, proposals, trust_events, credentials, approval_challenges, provider_events, card_events, ledger_entries, audit_events তৈরি। foreign key, unique bill reference/period ও idempotency constraint দিন। synthetic seed data রাখুন।

ধাপ ২ · Advisor: CSV preview/column mapping, error row report, Decimal-এ মাসিক মোট, category ও recurring detector। সংখ্যার প্রত্যাশিত ফল হাতে যাচাই। AI থাকলে শুধু ভাষা/অনিশ্চিত শ্রেণি প্রস্তাব।

ধাপ ৩ · Bills: manual entry দিয়ে শুরু; পরে OCR/AI extraction। safe date গণনা, duplicate check ও reminder schedule। worker পুনরায় চালালে একই বিলের দ্বিতীয় proposal নয়।

ধাপ ৪ · Trust: shadow decisions জমা, category-wise Wilson score, tier update। replay আলাদা badge ও source ID সহ। unsafe approval suggestion-এ তাৎক্ষণিক tier 0।

ধাপ ৫ · Guardian: পর্ব ৭-এর checks ছোট function-এ লিখুন। raw bill text নয়, validated fields input। policy version রেকর্ড।

ধাপ ৬ · AI adapter: structured JSON schema; অজানা field reject। prompt injection test করুন। AI unavailable থাকলে manual data ও deterministic Guardian দিয়ে demo চলবে।

> **শুরু করার সহজ পথ:** প্রথমে ১টি প্রাপক, ১টি ৳১,২০০ বিল, ৳৫,০০০ demo balance এবং manual approval end-to-end চালান; তারপর AI, CSV, mock card events ও monthly report যোগ করুন। বাস্তব provider key ছাড়া prototype সম্পূর্ণ রাখুন।

১১.১ · নতুন ফিচার বানানোর ক্রম

1. **Card feed mock:** merchant, timestamp, ৳ amount, currency, card token, provider event ID, pending/posted/refund নিয়ে signed sample event। একই ID-তে upsert; raw PAN/CVV নয়।
2. **Expenditure sync:** card purchase-কে expense; refund-কে offset; credit-card bill repayment-কে transfer হিসেবে চিহ্নিত করুন, নতুন খরচ নয়। user correction ও category learning রাখুন।
3. **Monthly job:** মাসের শেষ দিনে posted transactions reconcile, category total, budget comparison, anomalies ও subscription report। AI শুধু গণনাকৃত JSON-এর ওপর বাংলা explanation, সাশ্রয়/ব্যয় বাড়ানোর কারণ ও আয় বৃদ্ধির ছোট বাস্তব option দেয়।
4. **Risk + second device:** নতুন প্রাপক, অস্বাভাবিক অঙ্ক, নতুন device/স্থান, একাধিক ব্যর্থ চেষ্টা ইত্যাদি score; high risk-এ নিবন্ধিত অন্য ডিভাইসে নির্দিষ্ট amount/payee সহ push। "এটা আমার নয়" = app payment cancelled + bank fraud channel notice।
5. **Provider adapter:** প্রথমে fake bank; পরে provider sandbox; শেষে bank/PSP-এর লিখিত অনুমতি ও পরীক্ষার পর live। Submit, status, webhook signature, reversal ও dispute flow আলাদা করুন।

> একই device-এ দ্বিতীয় notification পাঠালে "অন্য ডিভাইস" যাচাই হয় না। আগে থেকে trusted device enrolment ও recovery নিয়ম বানাতে হবে। Push না পৌঁছালে ঝুঁকিপূর্ণ পেমেন্ট pending থাকবে।

১২ · approval, UI ও deployment

ধাপ ৭ · Passkey registration: লগইন করা ব্যবহারকারী server challenge নিয়ে platform credential নিবন্ধন করে। সার্ভারে credential ID/public key/user association রাখুন। জাল registration challenge ও ভুল origin reject করুন।

ধাপ ৮ · Payment approval: pending proposal খুলে বিস্তারিত দেখান; server challenge→transaction hash bind; browser assertion; সার্ভারে সব verification; demo execution; live mode-এ transactional outbox দিয়ে authorized provider request। ব্যবহারকারী cancel করলে pending বা cancelled, debit নয়।

ধাপ ৯ · ওয়েবসাইট: ৬টি প্রধান view - (১) খরচ ও Advisor, (২) বিল ও safe date, (৩) category-wise trust ladder, (৪) approval inbox, (৫) card activity + monthly advisor, (৬) history/audit। demo mode-এ "DEMO · no real money", partner sandbox-এ "SANDBOX", live mode-এ provider নাম ও status স্পষ্ট রাখুন।

ধাপ ১০ · Deploy: frontend ও backend HTTPS origin-এ; WebAuthn RP ID, allowed origin, session/cookie, CORS, CSRF এবং DB persistence কনফিগার। SQLite single-instance demo-তে ঠিক; বহু instance হলে transactional DB ব্যবহার করুন। scheduled worker আলাদা প্রক্রিয়ায়।

ডেমোর সাত দৃশ্য

CSV → Advisor; card purchase → auto expense; মাসিক report; নতুন বিল → safe date; shadow/replay → tier; approval inbox → fingerprint/PIN → SIM receipt; সন্দেহজনক লেনদেন → দ্বিতীয় ডিভাইসে deny → app block + audit। ভিডিওতে ডেমো সীমা ও track অবস্থান বলুন।

১৩ · Fingerprint থেকে bank request

```
def begin_approval(user, proposal_id):
    p = db.get_pending_owned(user, proposal_id)
    require((p.created_by_user or trust.allows_auto_prepare(p))
            and guardian.check(p).ok)
    tx_hash = sha256(canonical_payment(p))
    challenge = secure_random_32_bytes()
    db.bind_once(challenge, user, p.id, tx_hash, expires=120)
    return webauthn_options(challenge, userVerification="required")

def approve(user, proposal_id, assertion):
    with db.atomic_transaction():
        p = db.lock_pending_owned(user, proposal_id)
        c = db.lock_unused_challenge(user, p.id)
        require(not c.expired and c.tx_hash ==
                sha256(canonical_payment(p)))
        verify_webauthn(assertion, c.challenge, user.public_key,
                        ORIGIN, RP_ID, require_uv=True)
        require(guardian.check(p).ok)
        if risk.needs_second_device(p):
            require(second_device.verified_for_exact_tx(p))
        db.consume(c)
        db.mark_authorized(p)
        db.outbox_once(p.id, "SUBMIT_PAYMENT")

def payment_worker(p):
    # DB lock-এর বাইরে অনুমোদিত provider API; AI key পায় না
    result = provider.submit(p, idempotency_key=p.id)
    db.record_submission_or_unknown(p, result)
    # signed webhook/status query-তে confirmed/failed reconcile
```

এটি নকশার pseudocode। External API call DB transaction-এর ভেতর নয়। Timeout মানেই failure ধরে আবার নতুন payment নয়; একই idempotency key দিয়ে provider status জিজ্ঞাসা। provider-confirmed না হওয়া পর্যন্ত “paid” দেখাবেন না। Demo mode-এ provider adapter simulated ledger ব্যবহার করে।

১৪ · Threat model ও সৎ সীমাবদ্ধতা

ঝুঁকি	প্রতিরোধ / বাকি সীমা
ভুয়া বিল বা prompt injection	OCR/AI output data; allowlist, amount, duplicate, human review; সন্দেহে hold ও tier 0। নিখুঁত শনাক্তকরণ দাবি নয়।
Approval-এর পর payee/amount বদল	canonical hash→challenge binding, version lock, execution-এর আগে recheck।
Replay / double click / worker race	single-use expiring challenge, unique idempotency key, DB lock ও atomic transaction।
চুরি হওয়া session / device	HTTPS, secure cookie, session timeout, user verification; device PIN জানা অন্য ব্যক্তি ঝুঁকি।
AI ভুল অঙ্ক বা ভুল উপদেশ	Decimal/server হিসাব, schema validation, review; AI শুধু ব্যাখ্যা।
Server/DB compromise	least privilege, backups, audit hash chain; local audit নিজে অপরিবর্তনীয় নয়।
বাস্তব bank/PSP বা card API অপব্যবহার	prototype-এ কোনো live API/credential নয়। পরবর্তী integration-এ provider-এর লিখিত অনুমোদন, চুক্তি, security review ও প্রযোজ্য Bangladesh Bank শর্ত দরকার [৩]। Raw card PAN/CVV বা bank password সংগ্রহ নয়।

Card feed দেরিতে আসতে পারে; pending authorization, posted charge ও refund আলাদা। Suspicious alert কোনো অপরাধের চূড়ান্ত রায় নয়। বাংলাদেশে blockchain “কাজ করে না” এমন সার্বিক দাবি করা ঠিক নয়। এই প্রকল্পের অন-চেইন অংশ বাদ দেওয়ার কারণ: বাংলাদেশি বিল payment flow-এর সঙ্গে প্রাসঙ্গিকতা ও প্রতিযোগিতার নির্দিষ্ট track-এ fit।

১৫ · ন্যূনতম ২০টি acceptance test

#	পরীক্ষা	প্রত্যাশিত ফল
01	সঠিক CSV আমদানি	মোট ও category সঠিক
02	ভুল date/amount সারি	error report; অর্থ লেজারে নয়
03	পুনরাবৃত্ত subscription	মাসিক/বার্ষিক মোট সঠিক
04	AI বানানো অঙ্ক	server হিসাবই দেখায়
05	safe date ও buffer	সঠিক দিন গণনা
06	duplicate bill reference	দ্বিতীয় proposal নয়
07	অজানা প্রাপক	Guardian hold
08	অস্বাভাবিক 2× অঙ্ক	Guardian hold
09	অপর্যাপ্ত demo balance	debit নয়
10	জরুরি reserve ভাঙে	debit নয়
11	বিলের injection text	নিয়ম অপরিবর্তিত; audit
12	tier 0-তে auto prepare	নিষিদ্ধ
13	২০/২০, ৪০/৪০, ৮০/৮০	Wilson tier 1/2/3
14	unsafe AI suggestion	tier 0; pending বাতিল
15	high tier, assertion নেই	কোনো execution নয়
16	ভুল origin/RP ID বা UV=0	approval reject
17	expired/replayed challenge	reject; debit নয়
18	approval-এর পর অঙ্ক বদল	hash mismatch; reject
19	দুইবার click/worker race	একটি balanced receipt
20	overdue বা user cancel	alert; debit নয়

একই adversarial test একটি "শুধু LLM" baseline-এ চালান। কতটি unsafe proposal/approval হয়েছে, প্রকৃত সংখ্যা ও পদ্ধতি লিখুন; অনুমান নয়।

১৫.১ · Bank ও card integration test

#	পরীক্ষা	প্রত্যাশিত ফল
21	mock card purchase event	খরচের তালিকায় একবার যোগ
22	একই webhook পুনরায়	duplicate নয়
23	pending → posted → refund	মাসিক হিসাব সমন্বয়
24	credit-card bill payment	purchase-এর সঙ্গে double count নয়
25	unknown/timeout provider response	“paid” নয়; status reconcile
26	provider request retry	একই idempotency key, এক debit
27	suspicious app payment	hold + অন্য ডিভাইসে challenge
28	অন্য ডিভাইসে “আমার নয়”	app request block; audit + bank alert
29	বাইরের card charge, issuer API নেই	alert/review; block হয়েছে দাবি নয়
30	issuer card-control sandbox	provider-confirmed block status
31	শুধু account number যোগ	কোনো feed বা debit access নয়
32	POS purchase, issuer contract নেই	fingerprint gate দাবি নয়; feed থাকলে alert
33	issuer sandbox pre-authorization deny	issuer confirmed decline
34	online 3DS/OOB timeout	issuer সিদ্ধান্ত; app success নয়
35	cash expense হাতে যোগ	মাসিক মোটে একবার
36	bank consent প্রত্যাহার	sync/payment token অকার্যকর

Acceptance rule: “card event দেখা” ≠ “card authorization থামানো” । Sandbox demonstration-এ issuer/processor-এর mock সিদ্ধান্ত দেখান; বাস্তব ক্ষমতা partnership ছাড়া দাবি করবেন না।

১৬ · সময়রেখা, জমা ও প্রমাণ

দিন	অগ্রাধিকার
১৮ অক্টোবর	দল/track ঠিক; FastAPI+React skeleton; SQLite ledger; একটি bill → Guardian → pending। আয়োজককে Topic A/C/F fit ও বিস্তারিত requirement জিজ্ঞাসা।
১৯ অক্টোবর	Advisor, mock card event, monthly report, scheduler, Wilson trust, WebAuthn, ২০ core + integration test।
২০ অক্টোবর	HTTPS sandbox demo, suspicious alert + second-device rehearsal, audit, ভিডিও ও deck; দাপ্তরিক জমার নির্দেশে নির্দিষ্ট cutoff যাচাই করে আগেই জমা।

জমার evidence pack

- ইংরেজি ১ পৃষ্ঠার problem + scope: simulated BDT payments; no blockchain; selected topic-এর কারণ।
- Architecture/flow, Guardian policy, Wilson formula ও trust evaluation table।
- GitHub repo, README, synthetic demo data, deploy/run guide; গোপন key ছাড়া।
- ২০ test-এর pass/fail report, baseline comparison, নিরাপত্তা self-assessment।
- কার্ড ক্রয় → auto expense → মাসিক report এবং সন্দেহজনক payment → অন্য ডিভাইসের block decision দেখানো ভিডিও; ≤৫ মিনিট ডেমো ভিডিও ও ≈১০ মিনিট ইংরেজি deck (মূল গাইডের প্রস্তুতি লক্ষ্য; দাপ্তরিক জমার ফাইল সীমা যাচাই করুন)।
- SIM receipt, audit trail, failed injection ও tamper test-এর screenshot। এগুলো on-chain proof নয়।

> **শেষ checklist:** □ প্রত্যেক final payment-এ নতুন user verification □ payee/amount bound challenge □ no biometric capture □ demo-তে no live PSP call □ bank confirmation পৃথক □ card block-এর issuer dependency স্পষ্ট □ audit/receipt সঠিক লেবেল □ topic claim সৎ □ দল ও deadline যাচাই।

১৭ · যাচাইকৃত উৎস ও এখন কী করবেন

উৎস

1. **প্রতিযোগিতার আন্তর্জাতিক ট্র্যাক:** IEEE CIS Singapore, "2026 FinTechathon" - Topic A/C/F, দল ও সময়সূচি।
   https://www.ieee-cis-singapore.org/2026-fintechathon

2. **WebAuthn Level 3:** W3C - challenge, user verification, local biometric/PIN ও UV flag।
   https://www.w3.org/TR/webauthn-3/

3. **বাংলাদেশের PSP/PSO অনুমোদন:** Bangladesh Bank, "Approval Procedure of Payment System Operator / Payment Service Provider", এবং Bangladesh Bank-এর সাম্প্রতিক Payment Systems Report।
   https://www.bb.org.bd/aboutus/regulationguideline/psd/pso_psp_03022019.pdf
   https://www.bb.org.bd/pub/annual/psdreport/paymentreport_dec2025.pdf

4. **Issuer card alert/control-এর সীমা:** Visa Transaction Controls - issuer-নির্বাচিত alert/block, এবং commercial controls-এ issuer authorization।
   https://developer.visa.com/docs/vctc
   https://developer.visa.com/capabilities/visa-commercial-fraud-solutions/visa-commercial-payment-controls-vpc

উৎসগুলি ১৮ অক্টোবর ২০২৬-এ দেখা। প্রকাশিত সংক্ষিপ্ত প্রতিযোগিতা বিবরণ থেকে "real on-chain transaction hashes" বাক্যটির বিস্তারিত অফিসিয়াল requirement স্বাধীনভাবে নিশ্চিত করা যায়নি; মূল গাইডে এটি লেখা আছে। তবে Topic F যে on-chain, তা প্রকাশিত বিবরণে স্পষ্ট। কোনো দাবি জমা দেওয়ার আগে আয়োজকের মূল নথি/উত্তর মিলিয়ে নিন।

এখনই প্রথম তিন কাজ

1. দল ও জমার বিষয় ঠিক করুন; সাধারণ বাংলা বিল প্রকল্প হলে Topic A-র সঙ্গে fit পরীক্ষা করুন।
2. একটি synthetic bill, mock bank connector ও card purchase event দিয়ে Guardian→passkey→SIM receipt, খরচের auto-update ও মাসিক report চালান।
3. ২০ test-এর expected result আগে লিখে রাখুন; সফলতার সংখ্যা ও ব্যর্থতা সৎভাবে রিপোর্ট করুন।

> এই নথি নির্মাণ নির্দেশিকা। বাস্তব আর্থিক সেবা চালুর অনুমতি বা নিরাপত্তা সনদ নয়।